SOFTWARE ENGINEERING ARCHITECTURE

Modular Architecture Setup

Complete Blueprint for Building Scalable, Decoupled, and Maintainable Software Systems

Document Type: System Architecture Guide

Topic: Modular Architecture & Enterprise Project Setup

Target Audience: Software Architects, Technical Leads, Senior Developers

Status: Standard Reference

1. Introduction to Modular Architecture Setup

Modular architecture is a design pattern where an application is composed of distinct, self-contained, and independent modules. Setting up a modular architecture ensures that each domain feature can be developed, tested, updated, and deployed with minimal side effects on other parts of the system.

1.1 Why Choose Modular Architecture Setup?

As applications grow, monolithic codebases become fragile and difficult to maintain. A properly structured modular architecture solves this by enforcing strict domain boundaries and clear separation of concerns.

Core Advantages of Modular Setup:

- **Clear Ownership:** Engineering teams can take ownership of specific domain modules.
- **Parallel Development:** Multiple developers work simultaneously on distinct modules without file conflict.
- **Reusability:** Core utility modules (Auth, Logging, Notifications) can be reused across projects.
- **Easier Migration:** Modular monoliths can smoothly transition into microservices if needed.

1.2 Key Architecture Metrics

Metric	Ideal Target	Impact on Architecture
Cohesion	High Cohesion	Keeps strongly related business logic together inside the module.
Coupling	Low Coupling	Minimizes direct dependencies between different domain modules.
Encapsulation	Strict Scope	Exposes only clean public APIs while hiding internal implementations.

2. Standard Directory & Project Layout

A clean modular project layout structures files around business domain features rather than technical layers.

2.1 Enterprise Directory Tree

```
src/
├── app.ts                  # Main Application Entrypoint
├── config/                 # Global Environment Configuration
│   └── database.ts
├── modules/                # Feature Modules Directory
│   ├── auth/               # Auth Module
│   │   ├── auth.controller.ts
│   │   ├── auth.service.ts
│   │   ├── auth.repository.ts
│   │   ├── auth.routes.ts
│   │   └── index.ts        # Public Module Export
│   ├── user/               # User Module
│   │   ├── user.controller.ts
│   │   ├── user.service.ts
│   │   └── user.model.ts
│   └── payment/            # Payment Module
│       ├── payment.service.ts
│       └── payment.routes.ts
└── shared/                 # Shared Cross-Cutting Utilities
    ├── middleware/
    ├── utils/
    └── constants/
```

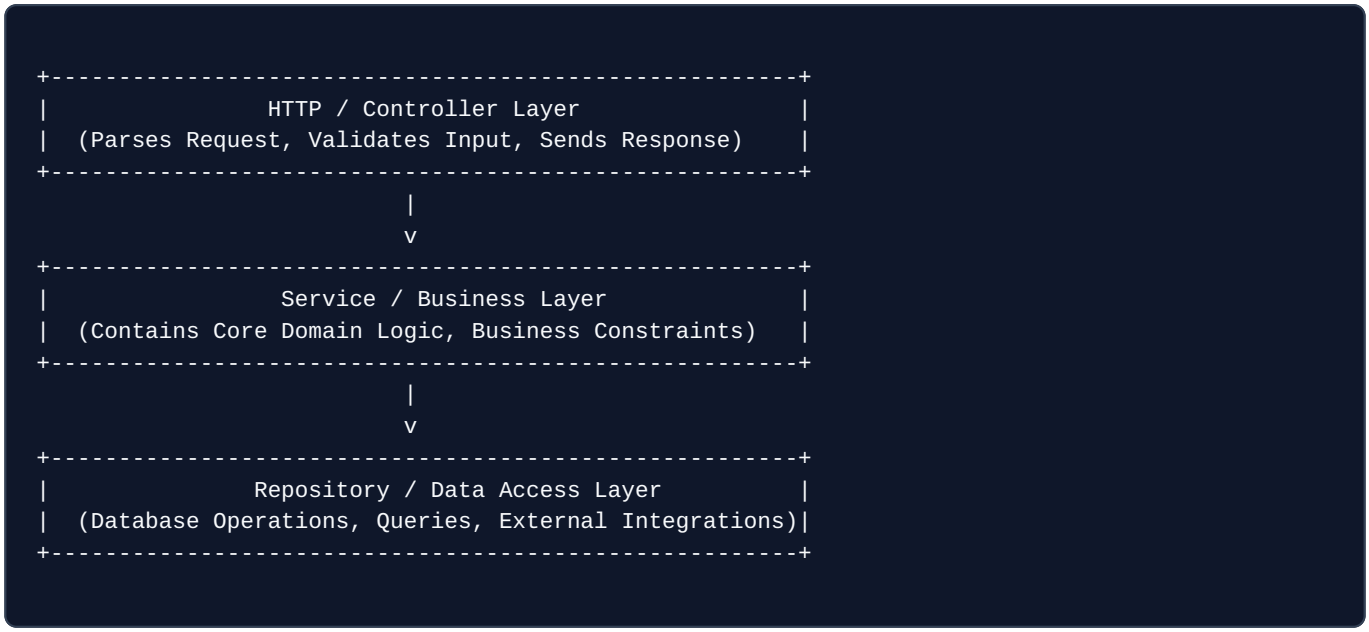
> **Key Convention:**
>
> Every module exposes a single `index.ts` (or public interface) file. External modules must only import from this public entry point, protecting private internal logic.

3. Domain-Driven Module Design (DDD)

Domain-Driven Design concepts help define clear bounded contexts for each module in the architecture.

3.1 Layered Module Architecture

Inside each individual feature module, logic is structured into distinct layers to separate business rules from transport protocols:



3.2 Layer Responsibilities

- **Controller:** Handles HTTP protocols, request payloads, and status codes.
- **Service:** Implements pure business requirements without HTTP dependency.
- **Repository:** Abstracts database ORM and data storage details.

4. Decoupling with Dependency Injection

To prevent modules from becoming tightly coupled, dependencies should be injected into modules via constructor parameters or interface contracts.

4.1 Code Implementation Example

```
// 1. Abstract Repository Contract
export interface IUserRepository {
  findById(id: string): Promise;
  save(user: User): Promise;
}

// 2. Service decoupled from DB implementation
export class UserService {
  constructor(private userRepository: IUserRepository) {}

  async getUserProfile(userId: string) {
    const user = await this.userRepository.findById(userId);
    if (!user) throw new Error("User not found");
    return user;
  }
}

// 3. Module Injection Wireup
const repo = new PostgresUserRepository();
const userService = new UserService(repo);
```

> **Testing Advantage:**
>
> Because `UserService` depends on the interface `IUserRepository`, we can inject a mock repository during unit tests without starting a real database server.

5. Inter-Module Communication Architecture

Direct function calls between modules can introduce tight coupling. Using an internal Event Emitter allows modules to communicate asynchronously.

5.1 Event Bus Pattern Setup

```
// Shared Event Bus Setup
import { EventEmitter } from 'events';
export const eventBus = new EventEmitter();

// Publisher: Auth Module (on user registration)
export class AuthService {
  async register(userData: RegisterDTO) {
    const newUser = await saveUser(userData);

    // Emit domain event without caring who listens
    eventBus.emit('user:registered', { userId: newUser.id, email: newUser.email });
    return newUser;
  }
}

// Subscriber: Notification Module
eventBus.on('user:registered', async (data) => {
  await sendWelcomeEmail(data.email);
});
```

Benefits of Event-Driven Coupling:

- Auth Module does not directly import Notification Module.
- New feature listeners (e.g., Analytics, Audit Logs) can be added without editing Auth code.

6. Shared Module Management

A common mistake in modular architecture is placing everything into a single bloated "shared" directory. Shared components must be managed strictly.

6.1 What Belongs in Shared Module?

Category	Allowed Shared Content	Forbidden Content
Utilities	Date formatters, Logger instance, Encryption helpers	Business-specific calculation logic
Middleware	Global Error Handler, CORS Setup, Request Logger	Domain-specific authorization rules
Types	Generic API Response DTOs, Base Entities	Domain-specific DB schemas

6.2 Guidelines for Shared Code

If two feature modules need to share specific business logic, create a dedicated new feature module instead of dumping code into the shared directory.

7. Frontend Modular Architecture Setup

Modular architecture applies equally to frontend applications (React, Angular, Vue) using feature-slice design.

7.1 Feature-Slice Layout

```
src/
├── features/
│   ├── auth/
│   │   ├── components/LoginForm.tsx
│   │   ├── api/authApi.ts
│   │   ├── hooks/useAuth.ts
│   │   └── index.ts
│   └── dashboard/
│       ├── components/WidgetGrid.tsx
│       └── index.ts
├── components/           # Reusable UI Atoms (Button, Modal)
└── pages/                # Route Page Composition
```

> **Rule of Encapsulation:**
>
> Components inside `features/auth/components` should not be imported directly by other features. Everything must go through `features/auth/index.ts`.

8. Testing & Build Strategy for Modules

A modular setup enables incremental testing and optimized CI/CD build pipelines.

8.1 Isolated Module Unit Testing

Each module contains its own test suites that execute independently:

```
// Running tests for specific module only
npm test -- --selectProjects=auth-module
```

8.2 Monorepo Tooling Integration

In large-scale enterprise applications, modular setups often use Monorepo tools (Turborepo, Nx, Lerna) to build only affected modules during pull requests:

- **Incremental Builds:** Cached builds skip modules that have no code changes.
- **Dependency Graph:** Visualizes relationships between internal modules.

9. Best Practice Summary & Setup Checklist

Setting up a modular architecture provides long-term maintainability, scalability, and code clarity.

9.1 Architecture Setup Checklist

1. Are modules organized around business features rather than technical file types?
2. Does each module expose a single, clean public entry point (e.g., `index.ts`)?
3. Are cross-module dependencies kept to a minimum using interfaces or event buses?
4. Is the shared folder restricted to generic, domain-agnostic utilities?
5. Are unit tests kept alongside their respective feature modules?

9.2 Architecture Comparison Matrix

Feature	Traditional Monolith	Modular Architecture
File Layout	Organized by controllers/services folders	Organized by business domain features
Coupling	High internal dependency risk	Low, enforced via public interfaces
Maintainability	Harder as codebase grows	High, isolated feature scopes
Scalability	Complex refactoring needed	Ready for microservice extraction

"A well-planned modular architecture setup turns complex applications into simple, manageable building blocks."